

Plateforme Mastering Live Debix

Architecture C — Spécification complète Phase 1a

Structure DSP, tests, livrables et procédure d'exécution

Document de référence technique

2026-04-21

Table des matières

1	Rappel : Architecture C	2
1.1	Décision validée	2
1.2	Principe directeur	2
1.3	Vue macro système	2
1.4	Contraintes héritées	2
2	Phase 1a — Objectifs et scope	3
2.1	Objectifs	3
2.2	Baseline existante — point de départ	3
2.3	Hors scope Phase 1a	3
2.4	Stratégie incrémentale d'exécution	3
3	Structure DSP détaillée	5
3.1	PCMs ALSA exposés	5
3.2	Inventaire des 6 pipelines SOF	5
3.3	Schéma détaillé des pipelines (Phase 1a complète)	5
3.4	Détail de la chaîne master (PIPE_3)	6
3.5	Détail du merge mux (PIPE_4)	6
4	Construction du fichier topology sof-imx8mp-tac5212-masterlite.m4	7
4.1	Principe	7
4.2	Mapping pipeline ↔ PCM	7
4.3	Squelette m4 complet (Phase 1a.1 MVP)	7
4.4	Pipelines m4 custom à créer (Phase 1a.1)	8
4.5	Évolutions 1a.2 et 1a.3	9
5	Budget latence	11
5.1	Chemins audio et latences attendues	11
5.2	Décomposition mic → HP direct (objectif 4 ms)	11
6	Tests T1–T9 — commandes complètes	12
6.1	T1 — Capture dry 8ch	12
6.2	T2 — Playback master	12
6.3	T3 — Master tap = master chain output	12
6.4	T4 — Null test bypass	12
6.5	T5 — Duplex stress 30 min	13
6.6	T6 — ALSA controls live	13
6.7	T7 — Bytes control mbdrc	13
6.8	T8 — Latence mic→HP ≤ 4 ms	13

6.9	T9 — Charge CPU DSP	14
6.10	Script unique <code>test-phase1a.sh</code>	14
7	Livrables — fichiers et leur localisation	15
7.1	Arborescence des fichiers Phase 1a	15
7.2	Fichiers SOF à créer / modifier	15
7.3	Fichiers Yocto à créer	16
8	Commandes de build et déploiement	17
8.1	Build topology uniquement (cas courant Phase 1a)	17
8.2	Build firmware SOF complet (cas où on modifie Kconfig ou sources)	17
8.3	Build complet Yocto (intégration dans l'image)	17
8.4	Deploy paquet seul sur la board	18
8.5	Deploy full image (rare, SD card)	18
9	Protocole d'exécution Phase 1a	19
9.1	Règles durables	19
9.2	Séquence 1a.1 — MVP	19
9.3	Séquence 1a.2 — Monitor direct	19
9.4	Séquence 1a.3 — Merge mux	19
9.5	Critères de sortie Phase 1a	19
10	Vue courte des phases suivantes	20
10.1	Phase 1b — Matrice complète	20
10.2	Phase 2 — USB gadgets	20
10.3	Phase 3 — Effets send (A53 JACK)	20
10.4	Phase 4 — NPU mastering	20
10.5	Phase 5 — Harmonizer + polish	20
11	Annexe — composants SOF mobilisés	21
11.1	Liste et rôles	21
11.2	Bypass bit-perfect — preuves code	21
11.3	ALSA controls exposés Phase 1a	21
11.4	Références datasheet TAC5212	21

1 Rappel : Architecture C

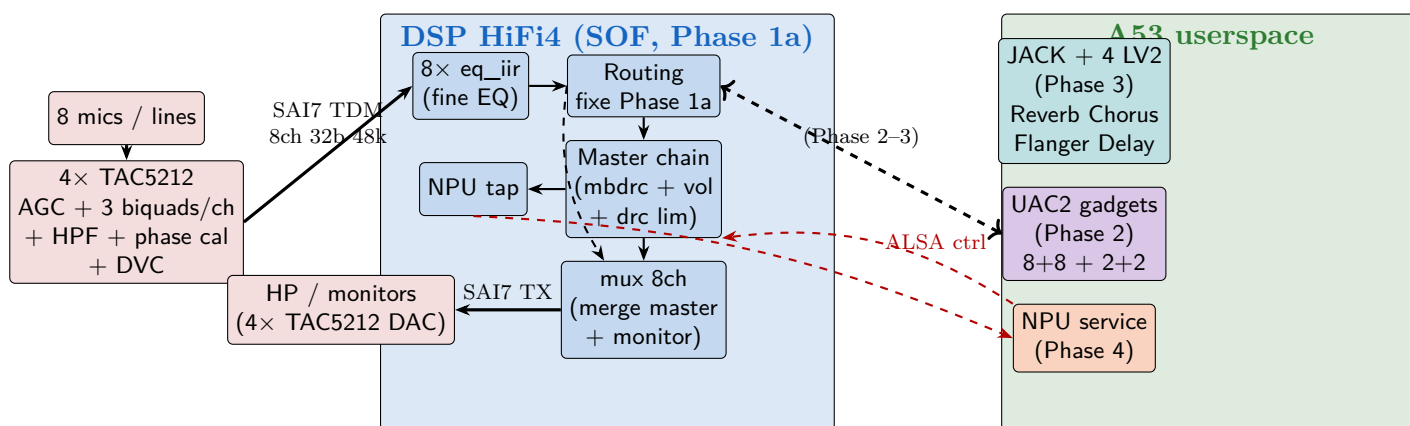
1.1 Décision validée

Architecture retenue par l'utilisateur (voir DSP_ARCHITECTURES.pdf — comparatif 3 archis). Architecture C dite "équilibrée" car elle répartit la charge entre les 3 étages (codec TAC5212, DSP HiFi4, Cortex-A53 userspace) selon leurs forces.

1.2 Principe directeur

- **Codec TAC5212** (I²C) : pré-conditionnement per-channel — AGC, HPF, biquads, gain/phase calibration, DVC. *Décharge le DSP du traitement par canal.*
- **DSP HiFi4 (SOF)** : ce qui DOIT être en basse latence — routage matrice, chaîne master stéréo, tap NPU. *Latence déterministe.*
- **A53 userspace** : USB gadget bridges, JACK+LV2 pour les effets send (reverb, chorus, flanger, delay), service NPU, UI Flutter. *Latence non critique.*

1.3 Vue macro système



En Phase 1a, seule la partie DSP (bleu) + TAC5212 (rouge) est implémentée. La partie A53 (vert) arrive en Phase 2+.

1.4 Contraintes héritées

Sample rate	48 kHz
Format	s32le
Nb canaux TDM	8 (TAC5212 ×4 daisy-chain)
DSP period SOF	2 ms
Latence mic → HP direct	≤ 4 ms
Latence round-trip USB (plus tard)	≤ 10 ms
IPC SOF	IPC3 (hérité platform i.MX8MP)
Null-test bypass	bit-perfect requis pour T4

2 Phase 1a — Objectifs et scope

2.1 Objectifs

1. Exposer 4 PCMs ALSA fonctionnels sur la carte SOF (softac5212tdm) :

Nom PCM	Type	Ch	Accès userspace typique
SAI_Capture	capture	8	arecord -D plughw:softac5212tdm,SAI_Capture
SAI_Playback	playback	8	aplay -D plughw:softac5212tdm,SAI_Playback
Source_Play	playback	2	aplay -D plughw:softac5212tdm,Source_Play
Master_Tap	capture	2	arecord -D plughw:softac5212tdm,Master_Tap

Ces **noms** sont explicitement déclarés dans la topology via PCM_DUPLEX_ADD / PCM_CAPTURE_ADD / PCM_PLAYBACK_ADD. On retrouve ces identifiants côté board dans arecord -l, /proc/asound/softac5212tdm/, et dans les scripts de test.

2. Implanter la **chaîne master stéréo** complète (multiband_drc + volume + drc limiter) avec modes bypass.
3. Garantir **latence mic** → **HP direct** ≤ 4 ms.
4. Fournir un **preset bypass bit-perfect** pour le null-test.
5. Exposer tous les paramètres via **ALSA controls** pour pilotage live (UI + NPU).
6. Configurer les 4 **codecs TAC5212** au boot (biquads flat, AGC off, HPF 12 Hz, DVC 0 dB).
7. Fournir un **script de test automatisé** couvrant T1 → T9.

2.2 Baseline existante — point de départ

La topology actuellement déployée sur la board fonctionne en duplex. Rappel :

- Fichier actif : sof-imx8mp-tac5212-drc2ms.m4 (variante 2 ms)
- Déployé en : /lib/firmware/imx/sof-tplg/sof-imx8mp-tac5212.tplg
- Validé utilisateur : sox -c 8 -t alsa plughw:2,0 -c 8 -t alsa plughw:2,0 remix 8 7 6 5 4 3 2 1 tourne en loopback 8 canaux sans clic ni xrun
- Structure : 1 pipeline capture avec DRC + 1 pipeline playback passthrough, exposé en 1 PCM duplex (ID 0)

La topology Phase 1a (sof-imx8mp-tac5212-masterlite.m4) **étend** cette baseline — elle garde le couple capture/playback SAI (qui marche) et **ajoute** les pipelines master chain + master tap.

2.3 Hors scope Phase 1a

- **Matrice N × M avec gain par cellule** → reporté Phase 1b.
- USB gadgets UAC2 → Phase 2.
- Effets send (reverb...) → Phase 3.
- NPU inférence + training → Phase 4.
- Harmonizer, Flutter UI avancée → Phase 5.

2.4 Stratégie incrémentale d'exécution

Phase 1a découpée en 3 étapes, **chacune validée par l'utilisateur avant la suivante** :

Étape	Livrables
1a.1 MVP	PIPE_1 (capture dry mics), PIPE_3+5 (Source_Play → master chain → Master_Tap), PIPE_6 (SAI_Playback 8ch direct). Permet tests T1, T2, T3, T4, T6, T7.
1a.2 Monitor	Ajout PIPE_2 (mic → HP direct 4 ms). Permet test T8 (latence).
1a.3 Merge	Ajout PIPE_4 via mux pour merge master (2ch, slots TDM 1-2) + monitor (6ch, slots 3-8). Phase 1a complète. Permet tests T5, T9.

3 Structure DSP détaillée

3.1 PCMs ALSA exposés

Côté host A53 (kernel ALSA), la carte `softac5212tdm` expose 4 devices PCM :

Nom PCM	Type	Canaux	Rôle
SAI_Capture	capture	8	Mics bruts post-TAC5212 ADC (avec eq_iir flat par défaut)
SAI_Playback	playback	8	Écriture directe vers TAC5212 DAC (monitoring depuis A53)
Source_Play	playback	2	Entrée stéréo master chain (source pour test <code>aplay</code>)
Master_Tap	capture	2	Sortie post master chain (NPU + null-test)

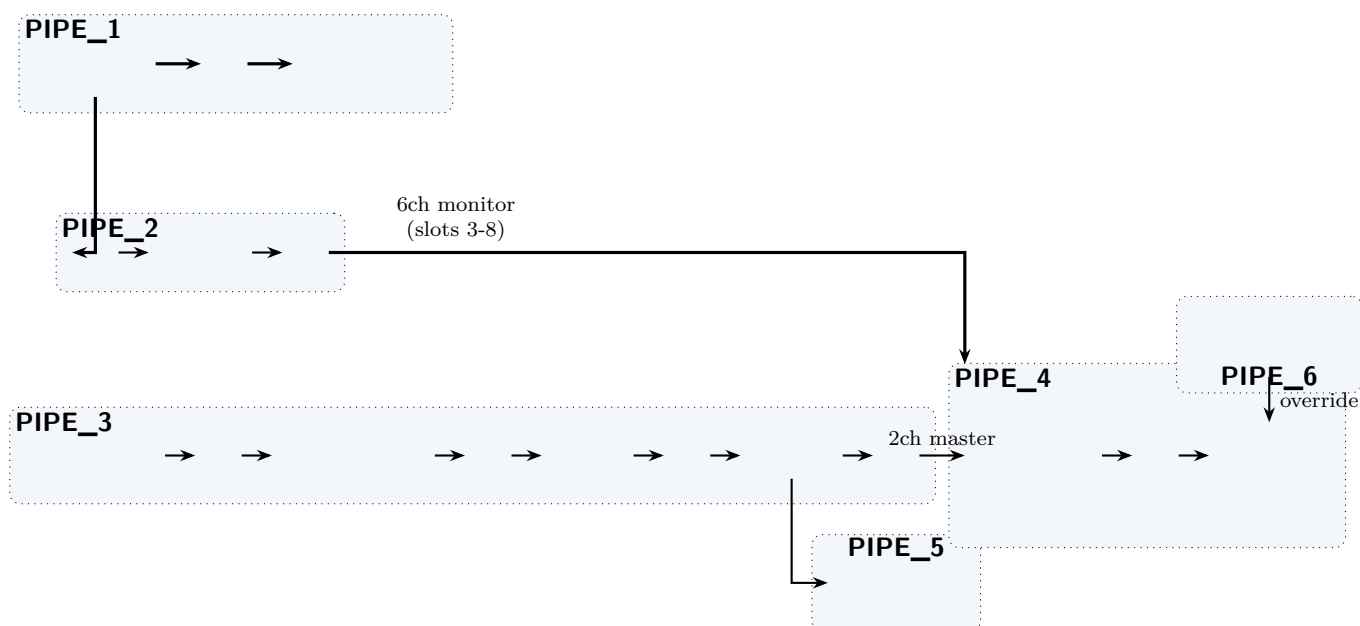
Un 5^e PCM `Source_Play` est ajouté en Phase 1a (vs. le plan initial 3 PCMs) pour permettre le test `aplay fichier.wav` sans signal mic live.

3.2 Inventaire des 6 pipelines SOF

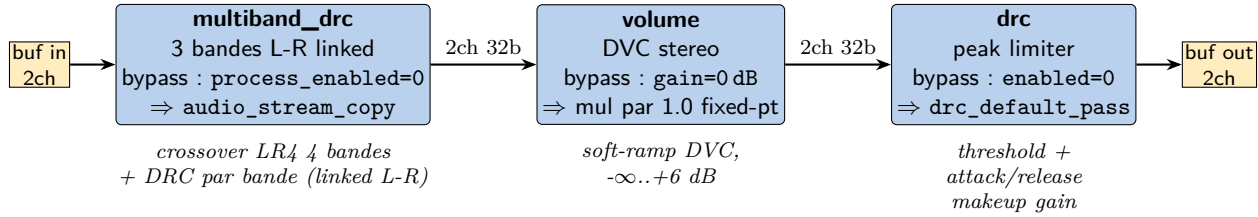
Pipeline	Période	Description
PIPE_1	2 ms	Capture dry — SAI7 RX → host <code>SAI_Capture</code> (8ch)
PIPE_2	2 ms	Monitor direct mic → HP low-latency 4 ms — SAI7 RX → eq_iir × 8 → PIPE_4 (via buffer)
PIPE_3	2 ms	Master chain stéréo — host <code>Source_Play</code> → multiband_drc → volume → drc → PIPE_4 et PIPE_5
PIPE_4	2 ms	Merge master + monitor — mux (2ch master sur slots 1-2 + 6ch monitor sur slots 3-8) → SAI7 TX (8ch)
PIPE_5	2 ms	Master tap host — PIPE_3 output (2ch) → host <code>Master_Tap</code>
PIPE_6	2 ms	Playback host direct — host <code>SAI_Playback</code> (8ch) → SAI7 TX (via mux slot override ou pipeline alternatif)

PIPE_6 est utilisé en 1a.1 (MVP) comme alternative simple à PIPE_4, puis remplacé/coexistant en 1a.3.

3.3 Schéma détaillé des pipelines (Phase 1a complète)



3.4 Détail de la chaîne master (PIPE_3)



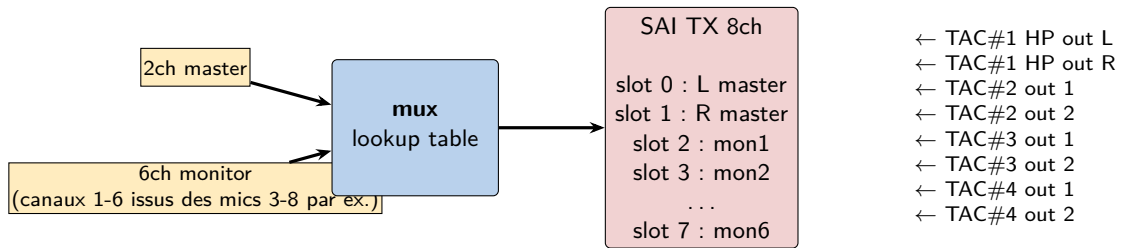
Chaque composant a un flag de bypass bit-perfect (prouvé par lecture du code) :

- multiband_drc_default_pass dans multiband_drc_generic.c:14 = audio_stream_copy(src,0,sink,0,c)
- drc_default_pass dans drc_generic.c:473 = idem
- volume à 0 dB (gain 0x40000000 en Q2.30) = multiplication par 1.0, pas de loss

Le null-test T4 compare la sortie de Master_Tap en mode bypass avec le signal envoyé à Source_Play. Différence attendue : **0 dBFS (silence parfait)**. Si ce n'est pas le cas, il y a un bug dans le pipeline ou un composant a un side-effect non documenté.

3.5 Détail du merge mux (PIPE_4)

Le composant mux (IPC3-compatible, voir src/audio/mux/mux.c) utilise une lookup table qui mappe des triplets (stream_id, in_ch, out_ch) vers le sink. Config Phase 1a :



Lookup table (sof_mux_config) :

stream_id	in_ch	out_ch	Commentaire
0	0	0	Master L → slot TDM 0
0	1	1	Master R → slot TDM 1
1	0	2	Monitor 1 → slot TDM 2
1	1	3	Monitor 2 → slot TDM 3
1	2	4	Monitor 3 → slot TDM 4
1	3	5	Monitor 4 → slot TDM 5
1	4	6	Monitor 5 → slot TDM 6
1	5	7	Monitor 6 → slot TDM 7

Stream 0 = sortie master chain (2ch). Stream 1 = 6 premiers canaux du monitor mic (on laisse tomber les 2 derniers mics sur ce merge, ou on garde les 8 via un mapping différent — décision Phase 1a.3).

4 Construction du fichier topology sof-imx8mp-tac5212-masterlite.m4

4.1 Principe

La topology Phase 1a est décomposée en 4 **pipelines** correspondant aux 4 PCMs. L'avantage : chaque pipeline est indépendant, on peut les activer/désactiver un par un pendant le développement.

4.2 Mapping pipeline ↔ PCM

Pipeline SOF	PCM exposé	PCM ID	Rôle
PIPE 1 + DAI_ADD capture	SAI_Capture	0 (via DUPLEX)	Dry mics 8ch
PIPE 6 + DAI_ADD playback	SAI_Playback	0 (via DUPLEX)	Direct 8ch vers DAC
PIPE 3 (master chain in)	Source_Play	1	Entrée stéréo master
PIPE 5 (master chain out)	Master_Tap	2	Sortie post-master (NPU)

4.3 Squelette m4 complet (Phase 1a.1 MVP)

Fichier : sof/tools/topology/topology1/sof-imx8mp-tac5212-masterlite.m4

```
#
# Topology Phase 1a MVP - 4 PCMs : SAI_Capture, SAI_Playback,
# Source_Play, Master_Tap + master chain stereo (mbdrc + volume + drc)
#
include('utils.m4')
include('dai.m4')
include('pipeline.m4')
include('sai.m4')
include('pcm.m4')
include('buffer.m4')
include('common/tlv.m4')
include('sof/tokens.m4')
include('platform/imx/imx8.m4')

# -----
# PIPE 1 : Dry mics capture (8ch) - baseline drc2ms equivalent
# -----
PIPELINE_PCM_ADD(sof/pipe-passthrough-capture.m4,
    1, 0, 8, s32le,
    2000, 0, 0,
    48000, 48000, 48000)

# -----
# PIPE 6 : Direct playback (8ch) - baseline drc2ms equivalent
# -----
PIPELINE_PCM_ADD(sof/pipe-passthrough-playback.m4,
    6, 1, 8, s32le,
    2000, 0, 0,
    48000, 48000, 48000)

# -----
# PIPE 3 : Master chain input side (HOST -> mbdrc -> volume -> drc -> buf)
# Utilise un pipeline CUSTOM local qui instancie la chaîne stéréo
# -----
PIPELINE_PCM_ADD(sof-imx8mp-tac5212-pipe-masterchain.m4,
    3, 2, 2, s32le,
    2000, 0, 0,
    48000, 48000, 48000)

# -----
# PIPE 5 : Master chain output side (buf -> HOST Master_Tap)
# Lit depuis le buffer de sortie de PIPE 3 via DAPM link
# -----
PIPELINE_PCM_ADD(sof-imx8mp-tac5212-pipe-mastertap.m4,
    5, 3, 2, s32le,
    2000, 0, 0,
    48000, 48000, 48000)

# -----
# DAI_ADD : SAI7 RX + TX (identiques a drc2ms.m4 qui fonctionne)
# -----
```



```

DAI_ADD(sof/pipe-dai-capture.m4,
        1, SAI, 7, tac5212-hifi,
        PIPELINE_SINK_1, 2, s32le,
        2000, 0, 0, SCHEDULE_TIME_DOMAIN_DMA)

DAI_ADD(sof/pipe-dai-playback.m4,
        6, SAI, 7, tac5212-hifi,
        PIPELINE_SOURCE_6, 2, s32le,
        2000, 0, 0, SCHEDULE_TIME_DOMAIN_DMA)

# -----
# PCM exports (les noms declares ici apparaissent dans /proc/asound/)
# -----
# PCM 0 duplex : SAI_Capture (cap) + SAI_Playback (play)
PCM_DUPLEX_ADD(TAC5212_SAI, 0, PIPELINE_PCM_6, PIPELINE_PCM_1)

# PCM 1 playback only : Source_Play (entree master chain)
PCM_PLAYBACK_ADD(Source_Play, 1, PIPELINE_PCM_3)

# PCM 2 capture only : Master_Tap (sortie post master chain)
PCM_CAPTURE_ADD(Master_Tap, 2, PIPELINE_PCM_5)

# -----
# Graph : connecter le buffer sink de PIPE 3 au source de PIPE 5
# (master chain output -> master tap input)
# -----
SectionGraph."master-chain-to-tap" {
    index "0"
    lines [
        'dapm(PIPELINE_SINK_5, PIPELINE_SOURCE_3)'
    ]
}

# -----
# DAI config SAI7 TDM 8 slots x 32 bits @ 48kHz (identique drc2ms.m4)
# -----
DAI_CONFIG(SAI, 7, 0, tac5212-hifi,
            SAI_CONFIG(DSP_A, SAI_CLOCK(mclk, 12288000, codec_mclk_in),
                        SAI_CLOCK(bclk, 12288000, codec_consumer),
                        SAI_CLOCK(fsync, 48000, codec_consumer),
                        SAI_TDM(8, 32, 255, 255),
                        SAI_CONFIG_DATA(SAI, 7, 0)))

```

4.4 Pipelines m4 custom à créer (Phase 1a.1)

Deux pipelines custom n'existent pas en upstream SOF et doivent être écrits dans `meta-local/recipes-kernel` (OU dans `sof/tools/topology/topology1/sof/` si acceptable). Règle : on ne touche PAS les upstream `sof/pipe-*.m4`.

sof-imx8mp-tac5212-pipe-masterchain.m4 Pipeline d'entrée de la chaîne master. Squelette :

```

# Master chain input: host PCM -> mbdrcc -> volume -> drc -> buf (shared)
include('utils.m4')
include('buffer.m4')
include('pcm.m4')
include('pipeline.m4')
include('bytecontrol.m4')
include('multiband_drc.m4') # multiband_drc widget macro
include('volume.m4')       # volume widget macro
include('drc.m4')          # drc widget macro

# Host PCM playback endpoint
W_PCM_PLAYBACK(PCM_ID, Source Play, 2, 0, SCHEDULE_CORE)

# Components
define(MBDRCC_priv, concat('mbdrcc_bytes_', PIPELINE_ID))
define(MBDRCC_CTRL, concat('mbdrcc_control_', PIPELINE_ID))
include('mbdrcc_coef_default.m4')
C_CONTROLBYTES(MBDRCC_CTRL, PIPELINE_ID, ..., MBDRCC_priv)
W_MULTIBAND_DRC(0, PIPELINE_FORMAT, 2, 2, SCHEDULE_CORE,
    LIST(' ', "MBDRCC_CTRL"))

W_VOLUME(0, PIPELINE_FORMAT, 2, 2, SCHEDULE_CORE, 0, 0)

```

```

define(DRC_priv, concat('drc_bytes_', PIPELINE_ID))
define(DRC_CTRL, concat('drc_control_', PIPELINE_ID))
include('drc_coef_limiter_default.m4')
C_CONTROLBYTES(DRC_CTRL, PIPELINE_ID, ..., DRC_priv)
W_DRC(0, PIPELINE_FORMAT, 2, 2, SCHEDULE_CORE,
    LIST(' ', "DRC_CTRL"))

# Buffers
W_BUFFER(0, COMP_BUFFER_SIZE(2, ...)) # host buffer
W_BUFFER(1, COMP_BUFFER_SIZE(2, ...)) # between mbdrc and volume
W_BUFFER(2, COMP_BUFFER_SIZE(2, ...)) # between volume and drc
W_BUFFER(3, COMP_BUFFER_SIZE(2, ...)) # OUTPUT (shared with PIPE 5)

# Graph: host -> B0 -> mbdrc -> B1 -> volume -> B2 -> drc -> B3
P_GRAPH(pipe-master-chain, PIPELINE_ID,
    LIST(' ',
        'dapm(N_BUFFER(0), N_PCM(PCM_ID))',
        'dapm(N_MULTIBAND_DRC(0), N_BUFFER(0))',
        'dapm(N_BUFFER(1), N_MULTIBAND_DRC(0))',
        'dapm(N_VOLUME(0), N_BUFFER(1))',
        'dapm(N_BUFFER(2), N_VOLUME(0))',
        'dapm(N_DRC(0), N_BUFFER(2))',
        'dapm(N_BUFFER(3), N_DRC(0))'))

# Export: buffer 3 est le SOURCE de la pipeline (visible au graph externe)
indir('define', concat('PIPELINE_SOURCE_', PIPELINE_ID), N_BUFFER(3))
indir('define', concat('PIPELINE_PCM_', PIPELINE_ID), Source Play PCM_ID)

# PCM capabilities
PCM_CAPABILITIES(Source Play PCM_ID, CAPABILITY_FORMAT_NAME(PIPELINE_FORMAT),
    PCM_MIN_RATE, PCM_MAX_RATE, 2, 2, 2, 16, 192, 16384, 65536, 65536)

```

sof-imx8mp-tac5212-pipe-mastertap.m4 Pipeline de sortie côté Master_Tap, simple capture depuis un buffer externe :

```

# Master tap: buf (shared) -> host PCM Master_Tap
include('utils.m4')
include('buffer.m4')
include('pcm.m4')
include('pipeline.m4')

# Host PCM capture endpoint
W_PCM_CAPTURE(PCM_ID, Master Tap, 0, 2, SCHEDULE_CORE)

# Buffer (sera lie au N_BUFFER(3) de la masterchain via DAPM externe)
W_BUFFER(0, COMP_BUFFER_SIZE(2, ...))

# Graph: buf -> host
P_GRAPH(pipe-master-tap, PIPELINE_ID,
    LIST(' ',
        'dapm(N_PCM(PCM_ID), N_BUFFER(0))'))

# Export: buffer 0 est le SINK de la pipeline (attendu par le graph externe)
indir('define', concat('PIPELINE_SINK_', PIPELINE_ID), N_BUFFER(0))
indir('define', concat('PIPELINE_PCM_', PIPELINE_ID), Master Tap PCM_ID)

PCM_CAPABILITIES(Master Tap PCM_ID, CAPABILITY_FORMAT_NAME(PIPELINE_FORMAT),
    PCM_MIN_RATE, PCM_MAX_RATE, 2, 2, 2, 16, 192, 16384, 65536, 65536)

```

4.5 Évolutions 1a.2 et 1a.3

1a.2 — Ajout PIPE 2 (monitor direct mic → HP)

- Nouveau pipeline sof-imx8mp-tac5212-pipe-monitor.m4 : SAI RX → eq_iir×8 → buf partagé avec PIPE 4
- Attach au scheduling de PIPE 1 via PIPELINE_SCHED_COMP_1 (pattern kwd)
- Ajouter dans le graph : dapm(PIPELINE_SINK_2, PIPELINE_SOURCE_1) pour partager SAI RX

1a.3 — Ajout PIPE 4 (mux merge master + monitor)

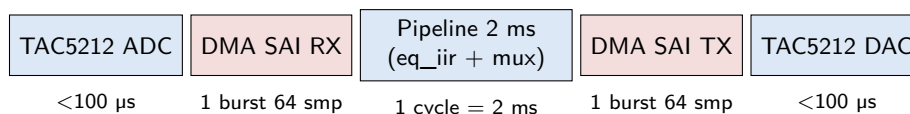
- Nouveau pipeline `sof-imx8mp-tac5212-pipe-muxout.m4` : 2ch master + 6ch monitor → mux → SAI TX
- Remplace ou coexiste avec PIPE 6 (SAI_Playback direct)
- Lookup table mux configurée via bytes control (cf. section 3.6 de ce document)

5 Budget latence

5.1 Chemins audio et latences attendues

Chemin	Latence	Contrainte
Mic → HP direct (PIPE_2 → mux → SAI TX)	4,0 ms	≤ 4 ms ✓
Mic → SAI_Capture host (PIPE_1)	4,0 ms	N/A
Source_Play → HP (PIPE_3 → PIPE_4)	8,0 ms	libre (playback mastering)
Source_Play → Master_Tap (PIPE_3 → PIPE_5)	8,0 ms	tolérable pour NPU (update 10 Hz)
Mic → Master_Tap (si matrice Phase 1b)	6–8 ms	tolérable

5.2 Décomposition mic → HP direct (objectif 4 ms)



Total ≈ 4 ms (2 cycles DSP period de 2 ms)

Marge de manœuvre : aucune. Si on ajoute un composant dans PIPE_2 (ex. compresseur per-ch), on dépasse 4 ms. Le fine-EQ par canal (3 biquads IIR) est acceptable car négligeable en temps de calcul ($\ll 100$ μ s par canal).

Si T8 échoue (mic→HP > 4 ms mesuré), mitigation :

- passer la DSP period à 1 ms → 2 ms total, mais charge CPU doublée
- retirer eq_iir de PIPE_2 et le faire uniquement dans PIPE_1 (capture) → perte d'EQ sur monitoring
- laisser TAC5212 faire toute l'EQ fine (3 biquads par canal en hard) et supprimer eq_iir du DSP

6 Tests T1–T9 — commandes complètes

Scripts et binaires stockés dans `/root/tests/` sur la board (convention projet). WAVs de référence générés avec `sox` sur la board.

6.1 T1 — Capture dry 8ch

But : vérifier que `SAI_Capture` expose bien les 8 canaux mics bruts, sans glitch.

```
# Génère un signal sur IN1 (ch1) via phone/PC branché
arecord -D plughw:softac5212tdm,SAI_Capture -c 8 -r 48000 -f S32_LE \
    -d 5 /root/tests/out/dry8.wav
sox /root/tests/out/dry8.wav -n stat # RMS / peak par canal
```

Pass si : ch1 a du signal (> -40 dBFS), ch2–8 au noise floor (< -100 dBFS), 0 xrun dans `dmesg`.

6.2 T2 — Playback master

But : jouer un tone stéréo à travers la chaîne master.

```
sox -n -r 48000 -c 2 -b 32 /root/tests/in/tone440.wav synth 3 sine 440 \
    vol -10dB
aplay -D plughw:softac5212tdm,Source_Play /root/tests/in/tone440.wav
```

Pass si : tone audible sur TAC#1 HP OUT, pas de clic, pas d’xrun.

6.3 T3 — Master tap = master chain output

But : vérifier que `Master_Tap` capte bien le signal traité.

```
arecord -D plughw:softac5212tdm,Master_Tap -c 2 -r 48000 -f S32_LE \
    -d 5 /root/tests/out/tap.wav &
sleep 1
aplay -D plughw:softac5212tdm,Source_Play /root/tests/in/tone440.wav
wait
sox /root/tests/out/tap.wav -n stat
```

Pass si : tap.wav contient le tone 440 Hz à niveau cohérent avec la chaîne (vol 0 dB + mbdrc & drc configurés).

6.4 T4 — Null test bypass

But : prouver que la chaîne master en bypass est bit-perfect.

```
# Activer bypass
amixer -c softac5212tdm cset name='Master_MBDRC_Bypass' on
amixer -c softac5212tdm cset name='Master_Volume' 0dB
amixer -c softac5212tdm cset name='Master_Limiter_Bypass' on

# Jouer et capturer
arecord -D plughw:softac5212tdm,Master_Tap -c 2 -r 48000 -f S32_LE \
    -d 5 /root/tests/out/bypass.wav &
sleep 1
aplay -D plughw:softac5212tdm,Source_Play /root/tests/in/tone440.wav
wait

# Null test : somme de tone440 + bypass inversé
sox -m -v 1 /root/tests/in/tone440.wav -v -1 /root/tests/out/bypass.wav \
    /root/tests/out/null.wav
sox /root/tests/out/null.wav -n stat 2>&1 | grep 'RMS.*amplitude'
```

Pass si : RMS de null.wav < -140 dBFS (bruit de quantification numérique). **Fail** si > -80 dBFS.

6.5 T5 — Duplex stress 30 min

But : stabilité sur long cours.

```
# Fichier de 30 min stéréo
sox -n -r 48000 -c 2 -b 32 /root/tests/in/stress.wav synth 1800 pinknoise vol -20dB
arecord -D plughw:softac5212tdm,SAI_Capture -c 8 -r 48000 -f S32_LE \
        -d 1800 /root/tests/out/stress_cap.wav &
arecord -D plughw:softac5212tdm,Master_Tap -c 2 -r 48000 -f S32_LE \
        -d 1800 /root/tests/out/stress_tap.wav &
aplay -D plughw:softac5212tdm,Source_Play /root/tests/in/stress.wav &
wait
dmesg | grep -i xrun | wc -l # doit retourner 0
```

Pass si : 0 xrun sur 30 min, stat des 3 fichiers cohérent.

6.6 T6 — ALSA controls live

But : modifier volume pendant playback sans glitch.

```
aplay -D plughw:softac5212tdm,Source_Play /root/tests/in/tone440.wav &
sleep 1
amixer -c softac5212tdm cset name='Master_Volume' -20dB
sleep 1
amixer -c softac5212tdm cset name='Master_Volume' 0dB
wait
```

Pass si : volume chute de 20 dB à l'oreille, remonte, pas de clic.

6.7 T7 — Bytes control mbdrc

But : pousser une config multiband_drc différente.

```
# Preset "heavy compression" préparé
sof-ctl -D softac5212tdm -n 'Master_MBDRC_Config' -b -s /root/tests/presets/mbdrc_heavy.bin

arecord -D plughw:softac5212tdm,Master_Tap -c 2 -r 48000 -f S32_LE \
        -d 5 /root/tests/out/tap_heavy.wav &
sleep 1
aplay -D plughw:softac5212tdm,Source_Play /root/tests/in/tone440.wav
wait

# Comparer dynamique
sox /root/tests/out/tap_heavy.wav -n stat | grep 'Maximum'
```

Pass si : peak du WAV "heavy" < peak "default" (compression plus forte).

6.8 T8 — Latence mic→HP ≤ 4 ms

But : mesurer physiquement la latence.

```
# Cable loopback : OUT1 (TAC#1 HP) --> IN2 (TAC#1 mic 2)
# Envoie un click (impulsion) via Source_Play, capture via SAI_Capture ch2

sox -n -r 48000 -c 2 -b 32 /root/tests/in/click.wav synth 0.001 square 1000 \
    vol -6dB pad 1 0

arecord -D plughw:softac5212tdm,SAI_Capture -c 8 -r 48000 -f S32_LE \
        -d 3 /root/tests/out/loop.wav &
sleep 0.1
aplay -D plughw:softac5212tdm,Source_Play /root/tests/in/click.wav
wait

# Calcule le delay par cross-correlation
python3 /root/tests/measure_latency.py /root/tests/in/click.wav \
    /root/tests/out/loop.wav --channel 1
```

Pass si : script Python reporte delay $\leq 4,2$ ms (marge 5%).

6.9 T9 — Charge CPU DSP

But : mesurer l'occupation CPU HiFi4 en fonctionnement nominal.

```
# Avec SOF-logger, lire les traces contenant CPC (cycles per chunk)
sof-logger -f /lib/firmware/imx/sof/sof-imx8m.ldc \
-o /tmp/sof.log -r 5 &
LOGPID=$!

# Charger le DSP : aplay + arecord + arecord master
aplay -D plughw:softac5212tdm,Source_Play /root/tests/in/stress.wav &
arecord -D plughw:softac5212tdm,SAI_Capture -c 8 -r 48000 -f S32_LE \
-d 10 /dev/null &
arecord -D plughw:softac5212tdm,Master_Tap -c 2 -r 48000 -f S32_LE \
-d 10 /dev/null &
wait

kill $LOGPID
grep CPC /tmp/sof.log | awk '{print_$NF}' | sort -u | tail
```

Pass si : utilisation rapportée $< 30\%$ de la capacité HiFi4.

6.10 Script unique test-phase1a.sh

Un script orchestre T1–T9 séquentiellement, produit un rapport Markdown. Exit 0 si tous passent.

7 Livrables — fichiers et leur localisation

7.1 Arborescence des fichiers Phase 1a

```
/home/michael/yocto-nxp-debix/
+-- sof/                                     # sous-module SOF, branche feature
|   +-- app/boards/imx8mp-evk-mimx8ml8-adsp.conf # override Kconfig si besoin
|   +-- tools/topology/topology1/
|       +-- sof-imx8mp-tac5212-masterlite.m4      # LA topology Phase 1a
|
+-- meta-local/
|   +-- recipes-kernel/linux/files/
|       +-- tac5212.c                             # driver codec existant
|       +-- tac5212-register-map.h                 # map des registres (Phase 1a)
|   +-- recipes-support/
|       +-- tac5212-init/
|           +-- tac5212-init_1.0.bb
|           +-- files/
|               +-- tac5212-init.c                 # service boot I2C
|               +-- tac5212-init.service           # systemd unit
|               +-- default.conf                   # preset biquad+AGC+DVC
|       +-- sof-presets/
|           +-- sof-presets_1.0.bb
|           +-- files/
|               +-- mbdrc_default.bin               # preset mbdrc par default
|               +-- mbdrc_heavy.bin                 # preset mbdrc compression forte
|               +-- drc_limiter_default.bin         # preset final limiter
|       +-- phase1a-tests/
|           +-- phase1a-tests_1.0.bb
|           +-- files/
|               +-- test-phase1a.sh                 # orchestrateur T1-T9
|               +-- measure_latency.py              # calcul delay par xcorr
|               +-- t1_capture_dry.sh               # T1 standalone
|               +-- t2_playback_master.sh
|               +-- ...
|   +-- recipes-core/images/
|       +-- imx-image-full.bbappend                # IMAGE_INSTALL += new packages
|
+-- PHASE_1A_SPEC.pdf                             # CE DOCUMENT
+-- PHASE_1A_SPEC.tex
+-- MASTERING_PLATFORM_PLAN.md                    # plan vivant
+-- DSP_ARCHITECTURES.pdf                         # comparatif A/B/C
```

7.2 Fichiers SOF à créer / modifier

Fichier	Action
sof/tools/topology/topology1/sof-imx8mp-tac5212-masterlite.m4	CRÉER (nouvelle topology Phase 1a)
sof/app/boards/imx8mp-evk-mimx8ml8-adsp.conf	MODIFIER (ajouter CONFIG_COMP_MULTIBAND_DRC=y et CONFIG_COMP_MUX=y si pas déjà présent ; laisser IPC3 par défaut)

Rappel règle projet : ne JAMAIS modifier les fichiers upstream `sof/tools/topology/topology1/sof/*.m4`. Si on a besoin d'un pipeline custom inconnu en amont, il vit dans `meta-local/recipes-kernel/linux/files/` sous un nom unique.

7.3 Fichiers Yocto à créer

Fichier	Rôle
<code>meta-local/recipes-support/tac5212-init/tac5212-init_1.0.bb</code>	Recette paquet service I ² C
<code>meta-local/recipes-support/tac5212-init/files/tac5212-init.c</code>	Service C — écrit les biquads/AGC/DVC au boot
<code>meta-local/recipes-support/tac5212-init/files/tac5212-init.service</code>	systemd unit (Type=oneshot, avant sof-firmware)
<code>meta-local/recipes-support/tac5212-init/files/default.conf</code>	Preset bi-quad/AGC/DVC au format texte
<code>meta-local/recipes-support/sof-presets/files/mbdrc_default.bin</code>	Bytes blob mbdrc (format SOF IPC3)
<code>meta-local/recipes-support/sof-presets/files/mbdrc_heavy.bin</code>	Preset compression forte (pour T7)
<code>meta-local/recipes-support/sof-presets/files/drc_limiter_default.bin</code>	Preset limiter -0,1 dBFS ceiling
<code>meta-local/recipes-support/phase1a-tests/files/test-phase1a.sh</code>	Orchestrateur de tests
<code>meta-local/recipes-support/phase1a-tests/files/measure_latency.py</code>	Script Python pour T8 (cross-correlation)
<code>meta-local/recipes-core/images/imx-image-full.bbappend</code>	IMAGE_INSTALL += tac5212-init sof-presets phase1a-tests

8 Commandes de build et déploiement

8.1 Build topology uniquement (cas courant Phase 1a)

Quand on modifie juste le .m4 de la topology, on ne reconstruit pas le firmware SOF complet — seulement le .tplg.

```
cd /home/michael/yocto-nxp-debix/sof/tools/topology/topology1

# 1. Preprocess m4
m4 -I . -I m4 -I common -I platform/common -I sof \
    sof-imx8mp-tac5212-masterlite.m4 \
    > /tmp/sof-imx8mp-tac5212-masterlite.conf

# 2. Compile conf -> tplg
alsatplg -c /tmp/sof-imx8mp-tac5212-masterlite.conf \
    -o /tmp/sof-imx8mp-tac5212-masterlite.tplg

# 3. Deploy (le kernel charge /lib/firmware/imx/sof-tplg/sof-imx8mp-tac5212.tplg)
scp /tmp/sof-imx8mp-tac5212-masterlite.tplg \
    root@192.168.0.9:/lib/firmware/imx/sof-tplg/sof-imx8mp-tac5212.tplg

# 4. Reboot (chargement tplg se fait au probe driver SOF)
ssh root@192.168.0.9 systemctl reboot

# 5. Verifier
ssh root@192.168.0.9 "dmesg | grep -i 'sof\|tplg' | tail -20"
```

Piège sed : si on crée une variante par sed 's/8000/4000/g', attention, le pattern matche aussi 48000 et 12288000. Toujours utiliser Edit avec un pattern précis.

8.2 Build firmware SOF complet (cas où on modifie Kconfig ou sources)

Si on ajoute CONFIG_COMP_MULTIBAND_DRC dans le board config, il faut rebuild le firmware .ri :

```
cd /home/michael/yocto-nxp-debix

# 1. Build firmware (Zephyr west)
west build -d /tmp/build-imx8m -b imx8mp-evk/mimx8ml8/adsp sof/app

# 2. Sign avec rimage
/tmp/build-rimage/rimage -o /tmp/build-imx8m/zephyr/zephyr.ri -e \
    -k sof/keys/otc_private_key.pem \
    -c sof/tools/rimage/config/imx8m.toml \
    /tmp/build-imx8m/zephyr/zephyr.elf

# 3. CRITICAL: concat xman + ri (sinon IPC 108/20 error)
cat /tmp/build-imx8m/zephyr/zephyr.ri.xman \
    /tmp/build-imx8m/zephyr/zephyr.ri \
    > /tmp/build-imx8m/zephyr/zephyr.ri.full

# 4. Deploy (fichier cible: sof-imx8m.ri, SANS 'p')
scp /tmp/build-imx8m/zephyr/zephyr.ri.full \
    root@192.168.0.9:/lib/firmware/imx/sof/sof-imx8m.ri

# 5. Reboot
ssh root@192.168.0.9 systemctl reboot

# 6. Verifier hash change
ssh root@192.168.0.9 "dmesg | grep 'Firmware info'"
```

8.3 Build complet Yocto (intégration dans l'image)

Pour les paquets Yocto (service tac5212-init, presets, tests) :

```

cd /home/michael/yocto-nxp-debix

# Initialiser environnement (une fois par session)
EULA=1 DISTRO=fsl-imx-xwayland MACHINE=imx8mpevk \
    source imx-setup-release.sh -b Model_AB_Infinity

# Rebuild d'un paquet uniquement
bitbake tac5212-init
bitbake phasela-tests
bitbake sof-presets

# Full image (pour deploy complet SD)
bitbake imx-image-full

# Rebuild complet si recette cassée
bitbake -c cleansstate <recipe> # JAMAIS cleanall (cf. memoire projet)
bitbake <recipe>

```

8.4 Deploy paquet seul sur la board

```

# Trouver le .deb produit
find Model_AB_Infinity/tmp/deploy/deb/imx8mpevk -name 'tac5212-init_*.deb'

# scp + install sur la board
scp Model_AB_Infinity/tmp/deploy/deb/imx8mpevk/tac5212-init_*.deb \
    root@192.168.0.9:/tmp/
ssh root@192.168.0.9 "dpkg_i_/tmp/tac5212-init_*.deb_&&_\
    systemctl_daemon-reload_&&_\
    systemctl_restart_tac5212-init"

```

8.5 Deploy full image (rare, SD card)

```

# Flash SD
dd if=Model_AB_Infinity/tmp/deploy/images/imx8mpevk/imx-image-full-imx8mpevk.wic \
    of=/dev/sdX bs=4M status=progress conv=fsync
sync

```

9 Protocole d'exécution Phase 1a

9.1 Règles durables

1. **Aucune modification de source SOF sans `critic_analyze` préalable.**
2. **Un changement = un commit = une validation utilisateur.**
3. **Backup systématique** avant toute édition d'un fichier existant (`cp fichier.m4 /tmp/fichier.m4.bak`).
4. **Jamais toucher `sof/tools/topology/topology1/sof/*.m4`** (fichiers upstream SOF). Nos pipelines custom vont dans `meta-local/` ou dans un fichier topology unique `sof-imx8mp-tac5212-masterlite.m4`.
5. **Tests stockés dans `/root/tests/`** sur la board, **jamais dans `/tmp/`** (mémoire projet).

9.2 Séquence 1a.1 — MVP

1. Design `sof-imx8mp-tac5212-masterlite.m4` minimal (`PIPE_1` + `PIPE_3` + `PIPE_5` + `PIPE_6`).
2. `critic_analyze` sur le diff topology.
3. Build topology (`m4` + `alsatplg`), deploy, reboot.
4. Exécuter T1 (capture dry).
5. Exécuter T2 (playback master avec preset par défaut).
6. Exécuter T3 (master tap reçoit bien le signal).
7. Exécuter T4 (null test bypass).
8. Exécuter T6 (`alsactrl` live).
9. Exécuter T7 (bytes control `mbdrc`).
10. Validation utilisateur → commit MVP.

9.3 Séquence 1a.2 — Monitor direct

1. Ajouter `PIPE_2` (SAI RX shared → `eq_iir×8` → buf partagé avec `PIPE_4`).
2. Pattern DAPM cross-pipeline (`PIPELINE_SCHED_COMP_1`) comme dans `kwd`.
3. `critic_analyze` sur le diff.
4. Build + deploy + reboot.
5. Exécuter T8 (latence mic→HP).
6. Validation utilisateur.

9.4 Séquence 1a.3 — Merge mux

1. Remplacer `PIPE_6` direct-playback par `PIPE_4` avec mux (2ch master + 6ch monitor).
2. Préparer la lookup table mux.
3. `critic_analyze`.
4. Build + deploy + reboot.
5. Exécuter T5 (stress 30 min) + T9 (CPU).
6. Validation utilisateur → Phase 1a OK, passage à Phase 1b.

9.5 Critères de sortie Phase 1a

- T1–T9 tous passent ;
- `test-phase1a.sh` exit 0 ;
- 0 xrun sur un run de 30 min ;
- Latence mic→HP ≤ 4 ms mesurée (T8) ;
- CPU DSP < 30% (T9) ;
- Documentation à jour : `MASTERING_PLATFORM_PLAN.md` marque Phase 1a comme *done*.

10 Vue courte des phases suivantes

10.1 Phase 1b — Matrice complète

Remplacer le routing fixe Phase 1a par une vraie matrice $N \times M$ via composants **mixer** parallèles (un par bus de sortie) + gains configurables via controls ALSA.

Pré-requis : valider Phase 1a stable + Phase 2 USB opérationnelle (car la matrice a plus de sens quand il y a plus d'inputs).

10.2 Phase 2 — USB gadgets

2 gadgets UAC2 via **configs** sur les deux contrôleurs USB du i.MX8MP :

- USB1 : 8in/8out pour DAW ASIO
- USB2 : 2in/2out pour iOS

Bridges ALSA entre les gadgets et le DSP via **alsaloop** (ou bridge C custom si latence serrée).

10.3 Phase 3 — Effets send (A53 JACK)

Serveur JACK2 avec 4 plugins LV2 (reverb, chorus, flanger, delay). Bridges ALSA $\leftrightarrow$ DSP via 4 PCMs capture + 4 PCMs playback supplémentaires.

Sélection des plugins à faire parmi :

- Reverb : **dragonfly-reverb**, **calf-reverb**
- Chorus : **calf-multichorus**
- Flanger : **calf-flanger**
- Delay : **x42-delay**, **calf-delay**

10.4 Phase 4 — NPU mastering

Training d'un modèle TFLite (from scratch) pour le mastering assisté :

- Input : FFT du master (2048 pts, 20 ms)
- Output : gains EQ 8-10 bandes + seuils mbdrc par bande + amount exciter
- Update rate : 10 Hz

10.5 Phase 5 — Harmonizer + polish

- Harmonizer pitch-shift : plugin LV2 **rubberband** chaîné en A53
- Exciter : plugin LV2 ou module SOF custom (à décider sur CPU budget)
- Flutter UI avancée : presets, scenes, métering en temps réel

11 Annexe — composants SOF mobilisés

11.1 Liste et rôles

Composant	Kconfig	Rôle Phase 1a
dai-copier	natif	Interface SAI7 RX/TX ↔ buffers DSP
host-copier	natif	Interface HOST PCM ↔ buffers DSP
eq_iir	COMP_IIR	3 biquads IIR/ch (fine EQ, flat par défaut)
mux	COMP_MUX	Merge 2ch master + 6ch monitor → 8ch SAI TX
multiband_drc	COMP_MULTIBAND_DRC	Compresseur multibande stéréo (bypass bit-perfect dispo)
volume	natif	Digital Volume Control master (0 dB bit-perfect)
drc	COMP_DRC	Final limiter (bypass bit-perfect dispo)

11.2 Bypass bit-perfect — preuves code

Composant	Fichier	Fonction bypass
multiband_drc	multiband_drc_generic.c:14	multiband_drc_default_pass ($\equiv$ audio_stream_copy)
drc	drc_generic.c:473	drc_default_pass ($\equiv$ audio_stream_copy)
volume	volume.c	gain Q2.30 = 0x40000000 = 1,0 $\Rightarrow$ multiplication par 1

11.3 ALSA controls exposés Phase 1a

Nom	Type	Plage / effet
SAI EQ Ch<N> Coeffs	bytes blob	3 biquads IIR (coefficients normalisés)
Master Volume	integer TLV dB	$-\infty..+6$ dB, 0.5 dB step
Master MBDRM Config	bytes blob	Structure sof_multiband_drc_config
Master MBDRM Bypass	switch 0/1	Active multiband_drc_default_pass
Master Limiter Config	bytes blob	Structure sof_drc_config
Master Limiter Bypass	switch 0/1	Active drc_default_pass

11.4 Références datasheet TAC5212

Voir tac5212.pdf dans la racine projet. Sections clés :

- §7.3.9.1 ADC signal chain (biquads, AGC, HPF, gain cal)
- §7.3.9.2 DAC signal chain (biquads, DRC DAC, volume, interpolation)
- §8.2 Programmable coefficient registers

*Document de référence Phase 1a. Tout écart à cette spec doit être tracé dans MASTERING_PLATFORM_PLAN.md et re-soumis à **critic_analyze**.*